

Highlights

A State-Machine Stackelberg Game for Diagnosis–Repair in Natural-Language Optimization

Kangzhen Peng

- Diagnosis–repair is cast as a Stackelberg inspection game defined on a LangGraph state machine as the stage-game apparatus.
- The diagnosis agent is an inspector that commits, first-mover, to a causal-order (data $\triangleright$ model $\triangleright$ code) probing policy with executed refutation.
- Accused agents are inspectees with a genuine deflection incentive; the inspector’s commitment induces truthful attribution.
- A proposition links causal-order inspection to a strictly smaller viable root-cause set than single-judge diagnosis.

A State-Machine Stackelberg Game for Diagnosis–Repair in Natural-Language Optimization

Kangzhen Peng^a

^a College of Transportation Engineering, Dalian Maritime University, No. 1 Linghai Road, Dalian, 116026, Liaoning, China

ARTICLE INFO

Keywords:

large language model
multi-agent system
Stackelberg game
finite-state machine
automated optimization model-
ing
empty container repositioning

ABSTRACT

We recast the error diagnosis and repair stage of an LLM multi-agent optimization pipeline as a Stackelberg *inspection game* defined on a state machine. A LangGraph finite-state machine orchestrates role-specialized agents (Data Engineer → Model Expert → Python Developer → Solver) and serves as the stage-game apparatus. When a solve fails, the diagnosis agent acts as an *inspector* (the Stackelberg leader) that commits, first and observably, to an inspection policy: a probing order following the pipeline's causal dependency (data ▷ model ▷ code) together with *executed refutation* — verifying every response by a real re-solve rather than by cheap talk. The accused agents are *inspectees* (the followers): because a downstream layer is routinely implicated by an upstream defect it did not cause, each accused agent has a genuine private incentive to deflect blame and avoid wasted repair, so its interests are not aligned with truthful attribution. We show that the inspector's committed causal-order, executed-refutation policy induces the truthful best response — a guilty layer complies and repairs, an innocent layer's deflection is verified — replacing the baseline's single-judge LLM accusation. We formalize the mechanism, state a proposition on causal-order inspection, and instantiate it for Empty Container Repositioning.

1. Introduction

This paper addresses the reliability of LLM multi-agent pipelines that automatically formulate and solve operations-research (OR) optimization problems from natural language. We inherit a working pipeline — the MAKO framework (Wu et al., 2024) — and focus on its weakest link: when the solver fails, attributing the failure to the correct pipeline stage and repairing it. We propose a state-machine Stackelberg game for this diagnosis–repair stage.

Thesis. Attributing a failed solve is not a neutral classification task but a game with conflicting interests, and the pipeline's causal layering (*data ▷ model ▷ code*) makes it precisely a Stackelberg *inspection game*. Because an upstream defect manifests as a downstream symptom, an accused downstream agent is often innocent, and it has a genuine private incentive to deflect blame upstream and avoid the cost of a repair that could not fix the true cause anyway. The diagnosis agent is therefore an *inspector* that moves first and commits — observably — to an inspection policy: probe the layers in causal order and verify every response by *executed refutation* (a real re-solve), not by an unverifiable confidence score. This commitment is what gives the mechanism teeth: it makes truthful compliance the best response of a guilty layer and verified deflection the best response of an innocent one, so the accused can neither confess falsely nor deflect dishonestly for free. Aligning the probing order with the causal order gives the first move causal meaning — a probe of a layer upstream of the true root cause is settled by re-solve before the inspector descends. The remainder of the draft develops the method (Section 3); Problem Definition, Experiments and Conclusion are placeholders pending the experimental campaign.

 pengkangzhen@dlmu.edu.cn (K. Peng)
ORCID(s):

2. Problem Definition

We adopt the three-echelon Empty Container Repositioning (ECR) problem of the baseline: a single-commodity, multi-period network flow on a directed graph $G = (N, E)$ whose nodes partition into sea ports H , dry ports L , consignees P and shippers Q , with arc set $\mathcal{A} \subseteq N \times N \times \Omega$ encoding admissible movements. The deterministic ground-truth instance used for evaluation has a known optimum, supplying the strict-success rule (OPTIMAL with optimality gap $\leq 10^{-3}$) and the root-cause labels required by the diagnosis mechanism below.

3. Proposed Method

This section develops the proposed framework bottom-up. We define the agents and their interfaces (Section 3.1), compose them into a state-machine architecture (Section 3.2), and specify their communication through a shared blackboard (Section 3.3). We then describe the incremental knowledge enhancement that grounds formulation decisions (Section 3.4). The final two subsections present the core contribution: a Stackelberg *inspection game* for diagnosis–repair defined *on* the state machine (Section 3.5) and its algorithmic realization (Section 3.6). Throughout, the framework targets *layered-error* natural-language optimization — OR pipelines with a data $\rightarrow$ model $\rightarrow$ code dependency in which errors propagate strictly downstream and contaminate downstream signals; we do not claim gains for general LLM reasoning.

3.1. Agent Definitions

We retain the baseline’s role-specialized agents. Let $\mathcal{A} = \{a_1, \dots, a_5\}$, with $a_1 = \text{DataEngineer}$, $a_2 = \text{ModelExpert}$, $a_3 = \text{PythonDeveloper}$, $a_4 = \text{SolverExecutor}$, $a_5 = \text{DiagnosisAgent}$. Each agent is characterized by a tuple

$$a_i = (i, \mathcal{R}_i, \mathcal{T}_i, \mathcal{B}_i, \mathcal{H}_i), \quad (1)$$

where i is a unique identifier, $\mathcal{R}_i$ the functional role, $\mathcal{T}_i$ the prompt template encoding role-specific instructions and output-format constraints, $\mathcal{B}_i$ an optional knowledge base (active only for a_2 , whose modeling decisions are domain-specialized), and $\mathcal{H}_i$ the hyper-parameter configuration (temperature, `top_p`, `max_tokens`).

The five roles decompose the modeling task along the causal order *data* $\triangleright$ *model* $\triangleright$ *code*. **DataEngineer** (a_1) parses the problem description and raw instance into structured parameters with symbol anchoring for downstream consistency; **ModelExpert** (a_2) designs the optimization model (decision variables, objective, constraints); **PythonDeveloper** (a_3) translates the formulation into executable solver code; **SolverExecutor** (a_4) runs the code in a sandbox, validates the result against the strict-success rule, and reports status; **DiagnosisAgent** (a_5) analyzes any failure and drives repair.

Each correctable agent exposes two maps: a *forward* map a_i^+ that produces its nominal output, and a *backward* (repair) map a_i^- that, given an error signal, either accepts responsibility and emits a corrected output or rebuts. Agent a_i receives an input assembled from prior outputs,

$$x^{(i)} \leftarrow f_i(y^{(0)}, y^{(1)}, \dots, y^{(i-1)}), \quad y^{(i)} \leftarrow a_i^+(x^{(i)}), \quad (2)$$

where f_i selects the necessary dependencies. The backward map is invoked only on the agent implicated by diagnosis (Section 3.5).

3.2. Multi-Agent Architecture and State-Machine Orchestration

The agents of Section 3.1 are composed into an extended finite-state machine rather than a free-form graph. The pipeline is compiled as

$$M = (\mathcal{Q}, q_0, \Delta, \mathcal{S}, K), \quad (3)$$

where $\mathcal{Q}$ is a finite set of control states (the graph nodes), q_0 the unique entry state, Δ the transition map, $\mathcal{S}$ the shared blackboard (Section 3.3), and K the transition budget. The state set is

$$\mathcal{Q} = \{q_{\text{DE}}, q_{\text{ME}}, q_{\text{KL}}, q_{\text{PD}}, q_{\text{SE}}, q_{\text{DG}}\} \cup \{q_{\text{DE}}^-, q_{\text{ME}}^-, q_{\text{PD}}^-\}, \quad (4)$$

comprising six forward states — one per agent in $\{a_1, \dots, a_5\}$ plus a dedicated KNOWLEDGELOADER state — and three backward (repair) states, one per correctable agent. The entry state is $q_0 = q_{\text{DE}}$ and the lone terminal is END. Figure 1 depicts M .

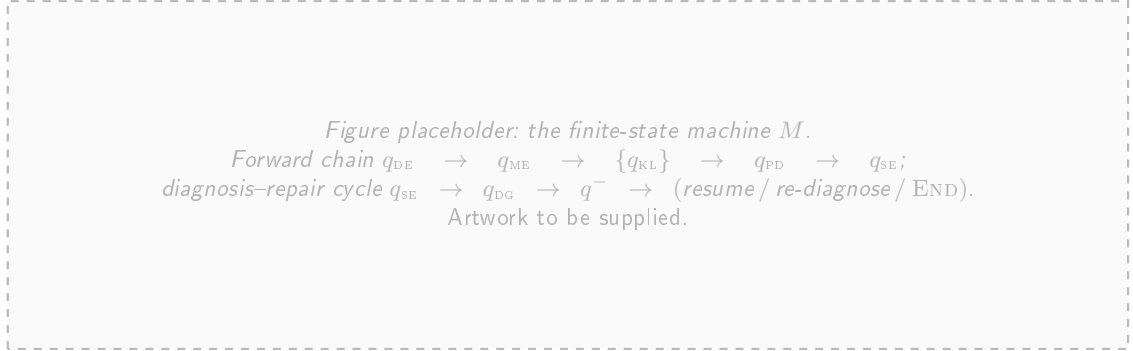


Figure 1: The extended finite-state machine $M = (\mathcal{Q}, q_0, \Delta, \mathcal{S}, K)$. Solid arrows: the forward chain with the incremental-knowledge loop (Section 3.4). Dashed arrows: the diagnosis–repair cycle, whose return edge is determined by the Stackelberg verdict (Section 3.5).

Transitions are pure functions of the blackboard. The transition map Δ is realized by five routing functions, each a pure map $\mathcal{S} \rightarrow \mathcal{Q}$:

- δ_{ME} : after ModelExpert, branch to KNOWLEDGELOADER iff it requests not-yet-loaded modules, else to PythonDeveloper — this edge realizes the incremental-knowledge loop of Section 3.4;
- δ_{KL} : after KnowledgeLoader, return to ModelExpert iff new knowledge was added (a refinement loop), else proceed to PythonDeveloper;
- δ_{SE} : after SolverExecutor, terminate on success or budget exhaustion, otherwise route to Diagnosis-Agent;
- δ_{DG} : after DiagnosisAgent, route to the backward state of the implicated agent (the diagnosis verdict);
- δ^- : after a backward step, either resume the pipeline downstream from the repaired agent, re-enter diagnosis, or terminate — depending on whether the repair was accepted and which agent was repaired.

The retry budget K (`max_retries`) bounds the number of diagnosis–repair cycles, i.e., it is the FSM’s transition budget over the diagnostic sub-graph.

The FSM is the stage-game apparatus, not scaffolding. The state machine is the structural substrate of the contribution because it is the object on which the repair game is defined. Its states are stage-game states, and — once the repair game is in play — the transitions δ_{DG} and δ^- become functions of the players’ strategy profile (the inspector’s committed inspection policy and the inspectee’s verified response; Section 3.5) rather than of a fixed heuristic. Treating M as the stage-game apparatus is what lets the commitment order carry *causal* meaning and what supports the proposition in Section 3.5; without this formalization, the FSM would be indistinguishable from generic graph orchestration.

3.3. Inter-Agent Communication via a Shared Blackboard

Agents never exchange messages pairwise. All communication is mediated by a single typed blackboard $\mathcal{S}$ (realized as the `AgentState` schema): every node reads its inputs from $\mathcal{S}$ and writes its outputs back to $\mathcal{S}$, and the transition functions Δ route solely on $\mathcal{S}$ ’s contents. This *shared-state, no-direct-links* design has three consequences.

Control and payload on one schema, separated by reader. $\mathcal{S}$ carries both *control state* and *data payload*. Control fields — retry count, the currently implicated agent, a resolved flag, the diagnosis mode, the round counter, the retry budget — drive Δ and are the only fields a routing function inspects. Data fields — each agent’s output, the assembled error evidence, execution metrics, the loaded knowledge modules, and the ground-truth objective — carry the pipeline’s content and are read by the agent that fires next. Colocating these concerns on one schema while separating their readers is what makes the FSM’s transitions both data-aware and cheap to reason about.

A consistent, fixed data view. A deterministic preprocessor runs once, before any LLM agent fires, and writes a canonical view of the raw instance into $\mathcal{S}$. Every downstream agent therefore reasons over an identical data basis, eliminating a class of data-parsing races that would otherwise masquerade as modeling or coding errors and confuse the diagnosis mechanism.

Append-by-field history. Communication is append-by-field rather than overwrite: a backward step writes a corrected output together with a resolved flag without erasing the prior forward outputs. The diagnosis mechanism can therefore inspect the full input–output history $\{(x^{(i)}, y^{(i)})\}$ when attributing a failure. This history is the evidence base $\mathcal{E}$ on which the Stackelberg game of Section 3.5 is defined, and its completeness is what makes blame attributable rather than heuristic.

3.4. Incremental Knowledge Enhancement

Mathematical modeling (a_2) is the most domain-specialized stage and the stage most prone to formulation errors, so `ModelExpert` is equipped with a domain knowledge base $\mathcal{B}_2$ of OR templates and solver APIs. Naive one-shot injection of all of $\mathcal{B}_2$ overloads the context window and dilutes attention; moreover, optimization modeling has rigid logical dependencies — a constraint cannot be formulated before its variables are defined — that semantic-similarity retrieval cannot respect. We therefore adopt an *incremental knowledge disclosure* (IKD) strategy: knowledge is loaded progressively, aligned with the modeling state.

Formally, instead of conditioning a_2 on $\mathcal{B}_2$ at once, we decompose the modeling trajectory and, at each step, apply a disclosure policy

$$\mathcal{B}_t = \pi(S_t, \mathcal{B}_2), \quad (5)$$

which selects the subset $\mathcal{B}_t \subset \mathcal{B}_2$ relevant to the current modeling state S_t (e.g. “variables defined, objective pending”). In the FSM this is realized by the $\delta_{\text{ME}}/\delta_{\text{KL}}$ loop: whenever `ModelExpert` requests a not-yet-loaded module, Δ routes to `KNOWLEDGELOADER`, the module is written into $\mathcal{S}$, and control returns to `ModelExpert` for a refinement pass; once no new module is requested, Δ proceeds to `PythonDeveloper`. IKD thereby reduces the conditional search space at each step and mitigates formulation hallucination — a prerequisite for clean attribution, since a knowledge-gap failure must not be confused with the modeling and code errors the diagnosis–repair game is designed to localize.

3.5. The Stackelberg Diagnosis–Repair Game as an Inspection Game

Why a game, not a single judge. The baseline diagnoses a failed solve with what it calls an “adversarial” protocol: (i) a static heuristic prior over candidate agents keyed on the solver status (`CRASH` $\Rightarrow$ code first; `OPTIMAL-but-wrong-objective` $\Rightarrow$ model first); (ii) a *single* LLM call returning a suspect, a confidence, and a rationale; (iii) a backward step in which the accused agent self-verifies. The baseline is candid that this is role-asymmetric but *not* game-theoretic — it “does not assume utility-maximizing agents, opposed payoff functions, or equilibrium behavior.” There is no payoff structure, no equilibrium concept, no leader–follower commitment, and no strategic interaction among agents.

We argue that the conflicting-interest structure is nonetheless already present and is being ignored. Under the causal layering of Section 3.5.1, an upstream defect surfaces as a downstream symptom, so the agent an observation implicates is frequently *not* the one at fault. A correctly self-interested accused agent should therefore *deflect* blame it does not deserve and decline a repair that cannot fix an upstream cause — exactly the incentive of an inspectee facing an inspector. We accordingly cast diagnosis–repair as a Stackelberg *inspection game* (Avenhaus et al., 2002; Maschler, 1966), a canonical class of leader–follower games in which an inspector commits to a monitoring policy and an inspectee best-responds. The diagnosis

agent is the inspector (leader); the accused correctable agents are inspectees (followers); and the inspector’s first-mover *commitment* to a causal-order, executed-refutation policy is what induces truthful attribution as the inspectees’ best response. Table 1 maps the inspection-game elements to the pipeline and contrasts them with the single-judge baseline.

Table 1

Mapping of Stackelberg inspection-game elements to the diagnosis–repair stage. Right column: what the single-judge baseline provides instead.

Inspection-game element	Concretization in our mechanism	Single-judge baseline
Inspector / leader (moves first, commits)	DiagnosisAgent commits an inspection policy σ : probe order + executed refutation	One LLM call picks a suspect
Inspectees / followers (best-respond)	Accused agents a_1, a_2, a_3 : comply-and-repair or deflect-upstream	Accused agent self-checks in isolation
Committed policy σ	Causal-order probing + verify by real re-solve (no cheap talk)	Fixed heuristic prior + single confidence
Conflict of interest	Innocent layer deflects to avoid wasted repair / wrongful blame	(assumed absent)
Inspector payoff	Correct root-cause localization at fewest rounds/tokens	(undefined)
Inspectee payoff	Avoid wrongful blame and wasted repair; fix only if truly at fault	A confidence score
Induced outcome	Guilty layer complies; innocent deflection is verified	Confidence-routed re-accusation
Stage-game apparatus	The LangGraph FSM (states = stage-game states)	(treated as plumbing)

3.5.1. Layered error dependence and the deflection incentive

Let the correctable core of the pipeline be the chain $a_1 \triangleright a_2 \triangleright a_3$ (DataEngineer $\triangleright$ ModelExpert $\triangleright$ PythonDeveloper), i.e. *data* $\triangleright$ *model* $\triangleright$ *code*, where $\triangleright$ denotes “is a causal prerequisite of.” We assume *strict layered error dependence*: a defect introduced at layer $\ell \in \{1, 2, 3\}$ may manifest at any layer $j \geq \ell$, but a defect at layer ℓ cannot be the root cause of a symptom that requires only layers $< \ell$ to be well-formed. Consequently, the observation of a downstream failure (e.g. an infeasible solve, a wrong objective) is consistent with several root-cause layers, and localizing the true root cause ℓ^* from the failure evidence $\mathcal{E}$ alone is fundamentally ambiguous. This ambiguity is exactly what a single-judge classifier must resolve in one shot.

The conflict of interest this creates. Layered dependence does not merely make attribution ambiguous; it gives the accused agents *misaligned private incentives*, which is what turns diagnosis into a game rather than a classification. Consider an agent at layer ℓ that a failure has implicated. Two facts hold simultaneously: (i) repair is costly — a backward step re-generates the agent’s output, consumes tokens, and risks injecting a fresh defect; and (ii) if $\ell \neq \ell^*$ (the layer is innocent, merely downstream of the true cause), then *no* local repair at ℓ can make the solve pass, because by strict layered dependence the true defect lies upstream. An innocent implicated agent therefore strictly prefers to *deflect* — to rebut the accusation and point blame upstream — rather than to accept a repair that wastes effort and cannot succeed. Symmetrically, even a guilty layer, if it could avoid the repair cost by deflecting, would be tempted to do so. The accused agents are thus not neutral truth-seekers whose objective coincides with the diagnostician’s; they are self-interested inspectees. A protocol that merely *asks* an accused agent to self-verify (the baseline) cannot separate a truthful deflection from a self-serving one, because both are cheap talk. Resolving this requires an inspector that *commits* to verification, which is what Section 3.5.2 constructs.

3.5.2. Players, strategies, and the inspector’s commitment

A diagnosis–repair stage game G is instantiated whenever Δ routes from q_{SE} to q_{DG} (the solver has failed and the budget is not exhausted). The game is played on the current blackboard $S_t \in \mathcal{S}$ and has the following structure.

Players. An *inspector* I (the leader) and a set of *inspectees* (the followers). The inspector is the DiagnosisAgent a_5 , operating as an independent examiner. The inspectees are the correctable agents a_1, a_2, a_3 ; in a given probe the active inspectee is the agent $a_{\hat{\ell}}$ at the currently probed layer $\hat{\ell}$.

The inspector’s strategy is a committed policy. The inspector’s strategy is an *inspection policy* $\sigma = (\omega, \nu)$ fixed *before* any inspectee responds and observable to all: (i) a probing order ω over the layers, and (ii) a verification rule ν that resolves every inspectee response by *executed refutation* — a real re-solve under the strict-success rule — rather than by accepting the inspectee’s word. Committing to ν is the crux: it converts the inspectee’s reply from cheap talk into a claim that will be mechanically checked.

The inspectee’s strategy is a response to being probed. When probed at layer ℓ , the inspectee chooses a response $r \in \{\text{COMPLY}, \text{DEFLECT}\}$. To COMPLY is to accept responsibility and emit a *concrete repair* $h \in \mathcal{H}_\ell$ — a specific fix with a justification tying it to the evidence — which the inspector will execute. To DEFLECT is to rebut the accusation and assert that the root cause lies at another (upstream) layer, a claim the inspector will likewise test.

Commitment order = causal order. The probing order ω follows the causal order $a_1 \triangleright a_2 \triangleright a_3$: the inspector probes the seeded suspect layer first and, when a layer is cleared, descends to the next causal layer. Play within a probe is strictly sequential — the inspector’s policy σ is committed and observable, then the inspectee best-responds — which is what makes G a Stackelberg game. Aligning the probing order with the causal order is the central design choice: it makes the first move causally meaningful (an upstream probe is settled by re-solve before the inspector can be blamed for descending too far) rather than a coin flip among observationally equivalent layers.

3.5.3. Inspectee response and verification by executed refutation

Inspectee response. Given the evidence $\mathcal{E}$ assembled from the blackboard — stack trace, solver log, the code segment implicated by the traceback, the model definition, and the relevant data structures, together with the input–output pairs $\{(x^{(i)}, y^{(i)})\}$ of the pipeline agents — the inspectee at the probed layer $\hat{\ell}$ chooses its response

$$(r, h) \leftarrow a_{\hat{\ell}}^-(x^{(\hat{\ell})}, y^{(\hat{\ell})}, \mathcal{E}), \quad r \in \{\text{COMPLY}, \text{DEFLECT}\}, \quad (6)$$

where, if $r = \text{COMPLY}$, $h \in \mathcal{H}_{\hat{\ell}}$ is a concrete repair with a justification ρ , and if $r = \text{DEFLECT}$, h carries a rebuttal pointing at an upstream layer. Complying is a bet that “the failure is caused by my layer, and here is precisely how I fix it”; deflecting is the claim “my layer is well-formed given its inputs; the cause is upstream.”

Verification by executed refutation. Under its committed rule ν , the inspector does not take the response on trust. It selects and *executes* the cheapest decisive test of the response,

$$\tau^* \leftarrow \arg \min_{\tau \in \mathcal{T}(r, h)} \text{cost}(\tau) \quad \text{s.t. } \tau \text{ is decisive}, \quad (7)$$

— a re-solve after applying h when $r = \text{COMPLY}$, or a targeted re-solve isolating the named upstream layer when $r = \text{DEFLECT}$ — and issues a verdict

$$v = (\beta, \hat{\ell}^*) = I(r, h, \tau^*, S_t), \quad (8)$$

where $\beta \in \{0, 1\}$ records whether the probed layer is *cleared* ($\beta=1$: a compliance whose re-solve still fails — a false confession — or a deflection the inspector upholds) or *confirmed* ($\beta=0$: a compliance whose re-solve passes the strict-success rule), and $\hat{\ell}^*$ is the inspector’s updated estimate of the true root-cause layer.

Executing the test rather than merely “reasoning about” it is what supplies a *grounded* signal: a re-solve either meets the strict-success rule or it does not, so neither a false confession nor a dishonest deflection survives verification.

Routing from the verdict. The FSM transition δ^- is a function of the executed verdict, not of a confidence score:

- if $\beta = 0$ (confirmed), the repair $\tilde{y}$ in h is committed and Δ resumes the pipeline *downstream* of $a_{\hat{\ell}}$;
- if $\beta = 1$ (cleared), Δ re-enters diagnosis and the inspector probes the next causal layer, until a confirmation or budget exhaustion (K).

This contrasts with the baseline, which routes on a single LLM confidence and re-accuses without an executed refutation.

3.5.4. Payoff structure and induced truthful attribution

Because LLMs admit no stable, observable utility function, we do *not* claim that an LLM “feels” a payoff. Instead we define numeric payoffs that the orchestrator optimizes and treat each LLM call as a *black-box best-responder*, analyzing the resulting trajectory. The inspector’s payoff rewards correct localization at low cost,

$$u_I(\sigma) = \mathbf{1}[\hat{\ell}^* = \ell^*] - \lambda_I \text{cost}(\sigma), \quad (9)$$

while the inspectee at layer ℓ , responding r , has the misaligned payoff

$$u_\ell(r; \sigma) = b \cdot \mathbf{1}[r=\text{COMPLY} \wedge \text{re-solve passes}] - c_{\text{rep}} \mathbf{1}[r=\text{COMPLY}] - c_{\text{blame}} \mathbf{1}[\text{held responsible} \wedge \ell \neq \ell^*], \quad (10)$$

where $b > 0$ is the reward for actually resolving the failure, $c_{\text{rep}} > 0$ the cost/risk of a repair, $c_{\text{blame}} > 0$ the penalty for carrying blame that does not fix the solve, $\text{cost}(\cdot)$ is token/round cost, and $\lambda_I \geq 0$ trades accuracy against budget. Eq. (10) makes the conflict explicit: the inspectee is rewarded only when it is *both* at fault *and* repairs, and is otherwise penalized for complying, so its interests diverge from the inspector’s whenever $\ell \neq \ell^*$. The ground-truth label ℓ^* enters only the *evaluation* of payoffs, never any agent’s prompt, so the game remains well-defined at inference time; equilibria are properties of the orchestrator-defined game, while each LLM is a sampled best-responder within it.

Commitment induces truthful attribution (the inspection-game deterrence effect). The value of the inspector’s first move is that its committed executed-refutation rule ν reshapes the inspectee’s best response. Consider the two cases. (i) *Innocent inspectee* ($\ell \neq \ell^*$): by strict layered dependence a local repair cannot make the solve pass, so COMPLY yields $-c_{\text{rep}}$ (no reward b , and it will still be seen not to fix the solve), whereas a truthful DEFLECT avoids the repair cost and is upheld by the inspector’s targeted re-solve. Deflection strictly dominates — and, crucially, only a *truthful* deflection survives ν , since a dishonest one is refuted by re-solve. (ii) *Guilty inspectee* ($\ell = \ell^*$): deflecting cannot succeed, because every upstream probe the inspector runs under causal order passes verification and the probe returns to ℓ^* having only consumed rounds; complying and repairing yields $b - c_{\text{rep}}$. Whenever $b > c_{\text{rep}}$ (a resolved solve is worth more than the repair), COMPLY is the guilty layer’s best response. Hence the committed policy makes *truthful attribution* — the guilty complies, the innocent deflects honestly — the induced best-response profile. This is the classical inspection-game deterrence result: it is the inspector’s *commitment*, not any assumed benevolence of the agents, that aligns individually optimal responses with correct root-cause localization. Without commitment (the baseline’s cheap-talk self-verification), deflection is free and truthful attribution is not incentive-compatible.

3.5.5. A proposition on causal-order inspection

Proposition 1 (Causal-order inspection lowers mis-attribution)

Under strict layered error dependence and an inspector committed to a causal-order, executed-refutation policy, each probe is *decisive*: it either confirms the probed layer (a compliance whose re-solve passes) or clears it (a false confession refuted, or a truthful deflection upheld). Consequently, causal-order inspection strictly reduces the per-round set of viable root-cause hypotheses relative to simultaneous (single-judge) play, and never eliminates the true layer. Formally, let V_{sim} and V_{ins} be the sets of root-cause layers that remain consistent with the failure observation after one round of, respectively, simultaneous diagnosis and causal-order inspection; then $V_{\text{ins}} \subseteq V_{\text{sim}}$ with $\ell^* \in V_{\text{ins}}$, and the inclusion is strict whenever a probed layer $\ell < \ell^*$ is cleared.

Sketch. Under simultaneous play the diagnostician selects a hypothesis without observing any committed, verified response, so every layer consistent with the downstream observation is a candidate; the viable set is the full observationally-equivalent upstream cone. Under committed inspection the inspectee at the probed layer ℓ must respond and the response is settled by executed refutation. If $\ell < \ell^*$, no local repair can pass the re-solve (strict layered dependence), so a compliance is refuted and a deflection upheld — either way ℓ is cleared and removed from the viable set. If $\ell = \ell^*$, the incentive analysis of Section 3.5.4 makes compliance the best response and the re-solve passes, so the true layer is confirmed and never wrongly eliminated. Causal ordering guarantees layers are cleared from the top of the upstream cone downward, so each cleared probe strictly shrinks V while simultaneous play can only guess within it. A formal bound on the mis-attribution rate and its empirical validation — centered on the commitment-order ablation — are deferred to the experimental section. $\square$

3.6. The Diagnosis–Repair Algorithm

Algorithm 1 unifies the architecture and the repair game. The FSM drives the forward pass together with the incremental-knowledge loop; on failure it enters the inspection stage, where the inspector commits to a causal-order, executed-refutation policy and probes layers in causal order — each probe eliciting an inspectee response and settling it by re-solve — until a layer is confirmed or the budget K is spent. The transition map Δ is exactly the five routing functions of Section 3.2, with the return edge out of the backward state set by the executed verdict v .

Relation to the baseline. The forward chain, the incremental-knowledge loop, the blackboard, and the retry budget are inherited unchanged. The replacement is localized: the single-judge diagnosis call and its confidence-routed return are substituted by the committed inspection stage, and the return edge of δ^- is driven by the executed verdict β rather than by a confidence. This locality is deliberate — it isolates the contribution (the inspection game) from the scaffolding (the FSM) and makes the commitment-order ablation, which randomizes the order in which layers are probed, the decisive experiment: if a permuted order performs as well as causal order, the thesis is falsified.

4. Experiments

[Placeholder] The experimental campaign evaluates root-cause attribution accuracy against the single-judge adversarial baseline and the sequential fallback, against external multi-agent debate (Du et al., 2023) and Reflexion-style self-critique (Shinn et al., 2023), with the commitment-order ablation (causal vs. permuted probing order) as the decisive test of Proposition 1. Inspection-specific diagnostics — the rate at which inspectees deflect, the fraction of deflections and confessions overturned by executed refutation, and the induced truthful-attribution rate as a function of the budget K — test whether the committed policy delivers the predicted deterrence effect (Section 3.5.4) rather than merely adding LLM calls.

5. Conclusion

[Placeholder] To follow the experiments.

Algorithm 1 FSM-Stackelberg diagnosis–repair (inspection game)

Require: Problem $\mathcal{P}$; agents $\{a_1, a_2, a_3, a_4\}$; inspector role a_5 ; FSM $M = (\mathcal{Q}, q_0, \Delta, \mathcal{S}, K)$; causal order $a_1 \triangleright a_2 \triangleright a_3$

Ensure: Result $Y \in \{\text{success}, \text{failure}\}$

```

1:  $S \leftarrow \text{PREPROCESS}(\mathcal{P})$ ;  $k \leftarrow 0$ ;  $Y \leftarrow \perp$ 
2: while  $Y = \perp$  do  $\triangleright$  drive the FSM
3:   // Forward chain:  $q_{DE} \rightarrow q_{ME} \rightarrow \{q_{KL}\} \rightarrow q_{PD} \rightarrow q_{SE}$ 
4:   for  $i = 1$  to 4 do
5:      $y^{(i)} \leftarrow a_i^+(S)$ ; update  $S$ ; advance  $\Delta$ 
6:   if  $y^{(4)}.\text{STATUS} = \text{OPTIMAL}$  then
7:      $Y \leftarrow \text{success}$ 
8:   else if  $k \geq K$  then
9:      $Y \leftarrow \text{failure}$ 
10:  else
11:    // Inspection stage at  $q_{DG}$ : inspector commits  $\sigma = (\text{causal order}, \text{executed refutation})$ 
12:     $\hat{\ell} \leftarrow \text{PRIOR}(y^{(4)}.\text{STATUS})$   $\triangleright$  seed probe order
13:    repeat
14:       $(r, h) \leftarrow a_{\hat{\ell}}^-(S)$   $\triangleright$  inspectee best-responds: comply-repair or deflect
15:       $\tau^* \leftarrow \arg \min_{\tau \in \mathcal{T}(r, h)} \text{cost}(\tau)$  s.t. decisive
16:       $(\beta, \hat{\ell}^*) \leftarrow a_5.\text{VERIFY}(r, h, \tau^*, S)$   $\triangleright$  executed refutation (re-solve)
17:       $k \leftarrow k + 1$ 
18:      if  $\beta = 0$  then  $\triangleright$  layer
19:         $\text{confirmedcommit } \tilde{y} \in h$ ; resume  $\Delta$  downstream
20:         $\hat{\ell} \leftarrow \text{NEXTCAUSALLAYER}(\hat{\ell})$   $\triangleright$  layer cleared: descend causal order
21:    until  $\beta = 0 \vee k \geq K$ 
21: return  $Y$ 
    
```

References

- Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM*, 2024.
- Y. Xiao et al. Chain-of-experts: When LLMs meet complex operations research. *arXiv preprint*, 2023.
- Y. Du et al. Improving factuality and reasoning in language models through multiagent debate. *arXiv:2305.14325*, 2023.
- N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- H. von Stackelberg. *The Theory of the Market Economy*. Oxford University Press, 1952.
- R. Avenhaus, B. von Stengel, S. Zamir. Inspection games. In R. J. Aumann and S. Hart (eds.), *Handbook of Game Theory with Economic Applications*, vol. 3, ch. 51, pp. 1947–1987. Elsevier, 2002.
- M. Maschler. A price leadership method for solving the inspector’s non-constant-sum game. *Naval Research Logistics Quarterly*, 13(1):11–33, 1966.
- M. L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *ICML*, 1994.